Hook: controllo deterministico in Claude Code e nell'Agent SDK

> Appunti per la certificazione CCA-F (Domini 1 e 3). Spiegano **cosa sono gli hook**, **quando usarli al posto del prompt** e **come funzionano** in Claude Code e nell'Agent SDK.
>
> ⚠️ Gli hook evolvono rapidamente. Per l'elenco aggiornato di eventi e campi fa fede la documentazione ufficiale: **docs.claude.com** → **Claude Code** → **Hooks** e **Agent SDK** → **Hooks**.

Indice

1. Il concetto chiave: prompt vs hook
2. Gli eventi
3. Come funziona un hook
4. Configurazione in Claude Code
5. Hook nell'Agent SDK
6. Pattern ricorrenti all'esame
7. Hook vs altri meccanismi
8. Trappole e domande da esame

---

1. Il concetto chiave: prompt vs hook

È la distinzione più importante dell'intero argomento, e all'esame ritorna di continuo.

	Istruzione nel prompt / `CLAUDE.md`	**Hook**
Chi la esegue	Il **modello**, che *dovrebbe* seguirla	Il **harness** (il runtime), sempre
Natura	**Probabilistica**	**Deterministica**
Garanzia	Migliora il comportamento, non lo garantisce	Garantisce il comportamento
Quando basta	Stile, preferenze, orientamento	Regole di business, sicurezza, compliance, validazione

> 💡 **Regola d'esame:** se lo scenario dice che una regola **deve** essere rispettata (policy, prerequisito, sicurezza) e i log mostrano che **a volte viene violata**, la risposta è quasi sempre un **hook** (o un controllo nel codice), non "rafforzare il prompt".

Esempi tipici:

- "Mai rimborsare prima di aver verificato cliente e ordine" → **PreToolUse** su `process_refund`.
- "Ogni citazione deve corrispondere alla fonte" → **PostToolUse** sul tool di estrazione.
- "Formatta sempre il codice" → **PostToolUse** su `Edit|Write` che lancia il formatter.

---

2. Gli eventi

Evento	Quando scatta	Può bloccare?	Uso tipico
`PreToolUse`	**Prima** dell'esecuzione di un tool	✅ Sì, impedisce la chiamata	Prerequisiti, comandi pericolosi, validazione input, permessi
`PostToolUse`	**Dopo** l'esecuzione di un tool	⚠️ Il tool è già stato eseguito; può però restituire un feedback bloccante a Claude	Validare l'output, formattare, lint, verifiche di provenienza
`UserPromptSubmit`	Quando l'utente invia un prompt	✅ Sì	Aggiungere contesto, filtrare prompt
`Stop`	Quando Claude sta per terminare	✅ Sì, può costringerlo a continuare	Lanciare i test prima di dichiarare "fatto"
`SubagentStop`	Quando un subagent termina	✅ Sì	Validare il risultato del subagent
`Notification`	Quando Claude invia una notifica	❌	Notifiche desktop / chat
`PreCompact`	Prima della compattazione del contesto	❌	Salvare informazioni importanti
`SessionStart`	All'avvio / ripresa della sessione	❌	Caricare contesto iniziale (es. issue aperte)
`SessionEnd`	Alla chiusura della sessione	❌	Pulizia, log

> 💡 **Pre vs Post:** `PreToolUse` *previene* (l'azione non avviene). `PostToolUse` *rileva* (l'azione è avvenuta, ma il risultato può essere rifiutato prima che venga usato a valle). Per azioni **irreversibili** (rimborsi, pagamenti, cancellazioni) serve `PreToolUse`.

---

3. Come funziona un hook

1. Il harness intercetta l'evento.
2. Se il `matcher` corrisponde (es. nome del tool), esegue l'hook.
3. L'hook riceve un **JSON su stdin** con i dettagli: `session_id`, `tool_name`, `tool_input`, e per `PostToolUse` anche `tool_response`.
4. L'hook risponde con un **exit code** e, opzionalmente, un **JSON su stdout**.

Exit code

Exit code	Effetto
`0`	OK, l'azione prosegue (lo stdout può aggiungere contesto in alcuni eventi)

Exit code	Effetto
`2`	**Blocca** l'azione; il messaggio su `stderr` viene restituito **a Claude**, che può correggersi
Altro	Errore non bloccante, mostrato all'utente

Output JSON (controllo più fine)

Per `PreToolUse` l'hook può decidere esplicitamente:

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason": "Rimborso bloccato: cliente non ancora verificato
con get_customer."
  }
}
```

`permissionDecision`	**Effetto**
`allow`	Consente senza chiedere conferma
`deny`	Blocca; il motivo arriva a Claude
`ask`	Chiede conferma all'utente

> 💡 Il **motivo** del blocco è fondamentale: deve dire a Claude **cosa fare** ("chiama prima `get_customer`", "usa `escalate_to_human`"), così l'agente si corregge invece di riprovare a vuoto.

---

4. Configurazione in Claude Code

Gli hook si definiscono in `settings.json` (user, project, local o managed).

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [{ "type": "command", "command": "python
.claude/hooks/block_dangerous.py" }]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [{ "type": "command", "command": "npx prettier --write ." }]
      }
    ],
```

```
    "Stop": [
      {
        "hooks": [{ "type": "command", "command": "npm test --silent" }]
      }
    ]
  }
}
```

- `matcher`: nome del tool o regex (`"Edit|Write"`, `"mcp__crm__.*"`). Vuoto o assente = tutti.
- Gli hook di progetto (`.claude/settings.json`) si committano: tutto il team ha le stesse garanzie.
- Nei **managed settings**, `allowManagedHooksOnly` impedisce agli utenti di aggiungere hook propri.
- Gli hook si possono definire anche nel frontmatter di **skill** e **subagent**, attivi solo mentre questi sono in uso.

Esempio di script `PreToolUse` che blocca comandi distruttivi:

```
import json, sys

data = json.load(sys.stdin)
cmd = data.get("tool_input", {}).get("command", "")
if "rm -rf" in cmd or "DROP TABLE" in cmd.upper():
    print("Comando distruttivo bloccato: chiedi conferma all'utente.",
file=sys.stderr)
    sys.exit(2)
sys.exit(0)
```

---

5. Hook nell'Agent SDK

Nell'Agent SDK gli hook sono **funzioni** registrate nelle opzioni dell'agente, non script esterni.

```
from claude_agent_sdk import ClaudeAgentOptions, HookMatcher

async def require_verification(input_data, tool_use_id, context):
    history = context.get("tool_history", [])  # stato tenuto dalla tua
applicazione
    verified = {"get_customer", "lookup_order"} <= {c["name"] for c in history if
c["ok"]}
    if not verified:
        return {
            "hookSpecificOutput": {
                "hookEventName": "PreToolUse",
                "permissionDecision": "deny",
                "permissionDecisionReason": "Prima verifica cliente e ordine con
get_customer e lookup_order.",
            }
        }
    return {}
```

```
options = ClaudeAgentOptions(
    hooks={"PreToolUse": [HookMatcher(matcher="mcp__support__process_refund",
hooks=[require_verification])]}
)
```

> ⚠️ L'esempio semplifica il tracciamento della cronologia: in pratica è la tua applicazione a registrare quali tool sono andati a buon fine (ad esempio con un `PostToolUse` che aggiorna lo stato).

Concetti da ricordare:

- Il nome dei tool MCP segue lo schema `mcp__<server>__<tool>`.
- Il blocco restituisce un **errore strutturato a Claude**, non interrompe l'intero agente: l'agente resta libero di gestire gli altri casi.
- Un `PostToolUse` sul tool **Task** (che lancia i subagent) permette di **validare il risultato di un subagent** prima che entri nel contesto del coordinatore.

---

6. Pattern ricorrenti all'esame

6.1 Prerequisiti di business (PreToolUse)

> "Il rimborso richiede cliente verificato e ordine trovato, ma nei log l'agente a volte chiama subito `process_refund`."

✅ Hook `PreToolUse` su `process_refund` che controlla la cronologia e restituisce un errore strutturato. ❌ Solo prompt più severo · ❌ pipeline fissa che toglie flessibilità · ❌ giochi con `tool_choice`.

6.2 Guardrail di rischio (PreToolUse + prompt)

> "Preservare la risoluzione al primo contatto, ma impedire rimborsi automatici in casi di frode."

✅ Prompt con **trigger osservabili** + hook `PreToolUse` che blocca il rimborso in presenza di segnali di rischio e indica di chiamare `escalate_to_human`. ❌ Approvazione umana su *ogni* rimborso (distrugge l'autonomia) · ❌ istruzione generica "fai attenzione alle frodi".

6.3 Validazione dell'output (PostToolUse)

> "Le citazioni estratte a volte non corrispondono alla pagina recuperata."

✅ Hook `PostToolUse` che confronta citazioni, URL e locator con il payload restituito, prima che il record venga accettato. ❌ Revisione del coordinatore alla fine (troppo tardi) · ❌ tool di audit chiamato "quando incerto".

6.4 Formato dei risultati dei subagent (PostToolUse su Task)

> "Il subagent a volte restituisce testo libero invece della struttura richiesta."

✅ Hook sul tool Task che fa il parsing, verifica i campi obbligatori e rifiuta o chiede riparazione. ❌ "Ignora le card malformate" in fase di sintesi · ❌ self-check nel prompt.

6.5 Verifica prima di dichiarare "fatto" (Stop)

✅ Hook `Stop` che lancia test o type-check e rimanda Claude al lavoro se falliscono.

---

7. Hook vs altri meccanismi

Esigenza	Meccanismo giusto
Vietare del tutto un tool a un agente	`allowedTools` / `disallowedTools` o regole `deny` nei permessi
Consentire un tool **solo a certe condizioni**	Hook `PreToolUse`
Controllare il **contenuto** del risultato di un tool	Hook `PostToolUse`
Guidare stile e preferenze	Prompt / `CLAUDE.md`
Forzare la chiamata di un tool specifico in un turno	`tool_choice` (API)
Validare la **forma** di un JSON	JSON Schema / structured output
Validare la **coerenza** con la fonte	Hook o codice applicativo che confronta con la fonte

> 💡 Permessi = **chi può** usare un tool. Hook = **quando e come** può usarlo.

---

8. Trappole e domande da esame

Trappole

- ❌ "Rafforza il system prompt" quando i log mostrano violazioni di una regola obbligatoria.
- ❌ Usare `PostToolUse` per impedire un'azione irreversibile: è già avvenuta.
- ❌ Un hook che blocca senza spiegare il motivo: l'agente non sa come correggersi.
- ❌ Bloccare tutto per sicurezza: l'hook deve colpire **solo** il caso a rischio.